

PROJECT DOCUMENTATION

Title: AI Chatbot using LangChain

1. Introduction

This project presents the development of a web-based AI chatbot that enables real-time conversational interaction using a Large Language Model (LLM). The system integrates LangChain for structured prompt orchestration and Groq's LLaMA 3.3 model for high-speed inference.

The application is built using Streamlit, which provides a lightweight and interactive user interface. The chatbot supports multiple chat sessions, allowing users to manage different conversations within a single interface.

2. Objectives

The primary objectives of this project are:

- To design and implement a real-time AI chatbot
- To integrate a high-performance LLM using the Groq API
- To utilize LangChain for prompt management and response generation
- To implement multi-session chat functionality
- To develop a clean and user-friendly interface using Streamlit
- To ensure secure handling of API credentials

3. Technologies and Libraries Used

3.1 Technologies

- Python: Core programming language
- Streamlit: Frontend interface development
- Groq API: LLM inference engine (LLaMA 3.3 70B)
- LangChain: Framework for chaining LLM operations

3.2 Libraries

- streamlit: UI rendering and interaction
- python-dotenv: Environment variable management
- langchain_groq: Integration with Groq models
- langchain_core.messages: Message structure handling
- langchain_core.prompts: Prompt template creation
- langchain_core.output_parsers: Response parsing

4. System Architecture

4.1 Workflow Description

The system follows a sequential pipeline:

User Input → Streamlit Interface → Session State Management → Prompt Template → LangChain Chain → Groq LLM → Output Parser → Response Display

4.2 Component Description

User Interface (Streamlit)

Handles user input, displays chat messages, and manages navigation between multiple chat sessions.

Session Management

Uses `st.session_state` to store chat history and manage multiple conversations dynamically.

Prompt Template

Defines system-level instructions and formats user queries before sending them to the LLM.

LangChain Pipeline

Combines prompt template, LLM, and output parser into a single execution chain:

- Input is structured
- Model processes the query
- Output is formatted for display

Groq LLM (LLaMA 3.3 70B)

Processes the input prompt and generates a natural language response with low latency.

Output Parser

Ensures that the response is converted into a clean string format before rendering.

5. Features

- Real-time AI chatbot interaction
- Multi-chat session support
- Automatic chat title generation based on user input
- Structured prompt handling using LangChain
- Clean and responsive user interface
- Secure API key management using environment variables

6. Code Implementation

```
import streamlit as st
```

```
import os

from dotenv import load_dotenv


# LangChain imports
from langchain_groq import ChatGroq
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser


# ----- LOAD ENV -----
load_dotenv()
api_key = os.getenv("GROQ_API_KEY")


if not api_key:
    st.error("Missing GROQ_API_KEY")
    st.stop()


# ----- LLM -----
llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    api_key=api_key
)


# ----- PROMPT TEMPLATE -----
prompt_template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful AI assistant. Give clear and concise answers."),
    ("user", "{question}")
])


# ----- CHAIN -----
chain = prompt_template | llm | StrOutputParser()
```

```
# ----- PAGE CONFIG -----

st.set_page_config(page_title="AI Chatbot", layout="wide")


# ----- SESSION -----

if "chats" not in st.session_state:
    st.session_state.chats = {}
    st.session_state.current_chat = None


# ----- SIDEBAR -----

with st.sidebar:
    st.title("💬 Chats")

    if st.button("➕ New Chat"):
        st.session_state.current_chat = None
        st.rerun()

    st.markdown("### Recents")

    for title in st.session_state.chats:
        if st.button(title, key=title):
            st.session_state.current_chat = title
            st.rerun()


# ----- CURRENT CHAT -----

if st.session_state.current_chat:
    messages = st.session_state.chats[st.session_state.current_chat]
else:
    messages = []


# ----- CSS -----
```

```
st.markdown("""  
<style>  
.chat-container {  
    max-width: 900px;  
    margin: auto;  
}  
.user-msg {  
    background-color: #2563eb;  
    color: white;  
    padding: 12px;  
    border-radius: 12px;  
    margin: 10px 0;  
    max-width: 70%;  
    margin-left: auto;  
}  
.bot-msg {  
    background-color: #1f2937;  
    color: white;  
    padding: 12px;  
    border-radius: 12px;  
    margin: 10px 0;  
    max-width: 70%;  
    margin-right: auto;  
}  
</style>  
""", unsafe_allow_html=True)
```

----- DISPLAY -----

```
st.markdown('<div class="chat-container">', unsafe_allow_html=True)
```

```
for msg in messages:
```

```

if isinstance(msg, HumanMessage):
    st.markdown(f'<div class="user-msg">{msg.content}</div>', unsafe_allow_html=True)
else:
    st.markdown(f'<div class="bot-msg">{msg.content}</div>', unsafe_allow_html=True)

st.markdown('</div>', unsafe_allow_html=True)

# ----- INPUT -----
user_input = st.chat_input("Type your message...")

if user_input:

    # Create new chat if none exists
    if st.session_state.current_chat is None:
        title = user_input[:40] + ("..." if len(user_input) > 40 else "")
        st.session_state.current_chat = title
        st.session_state.chats[title] = []

    # Save user message
    st.session_state.chats[st.session_state.current_chat].append(
        HumanMessage(content=user_input)
    )

    try:
        # 🔥 Use chain instead of raw LLM
        response = chain.invoke({"question": user_input})

        st.session_state.chats[st.session_state.current_chat].append(
            AIMessage(content=response)
        )

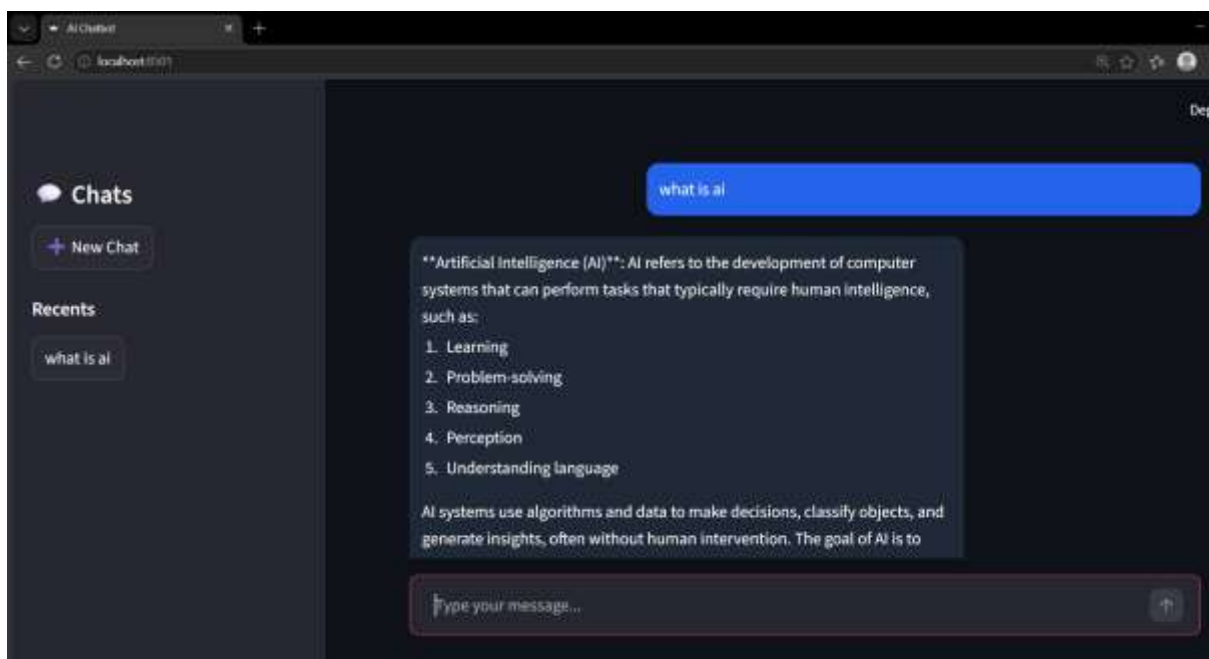
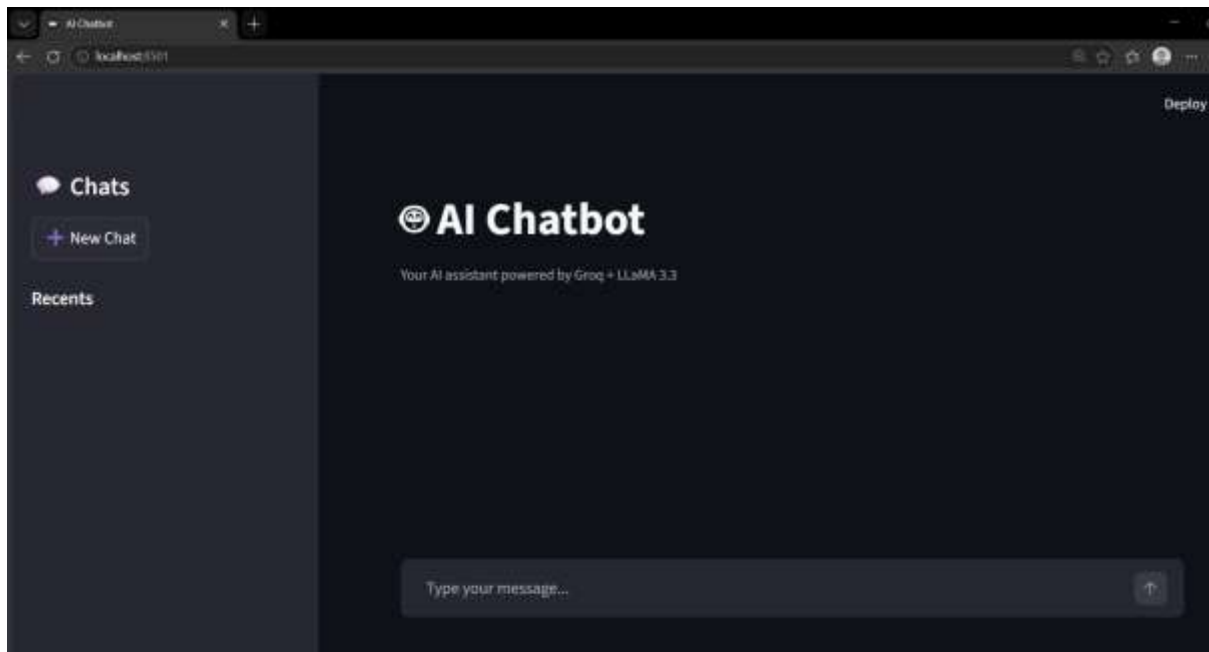
```

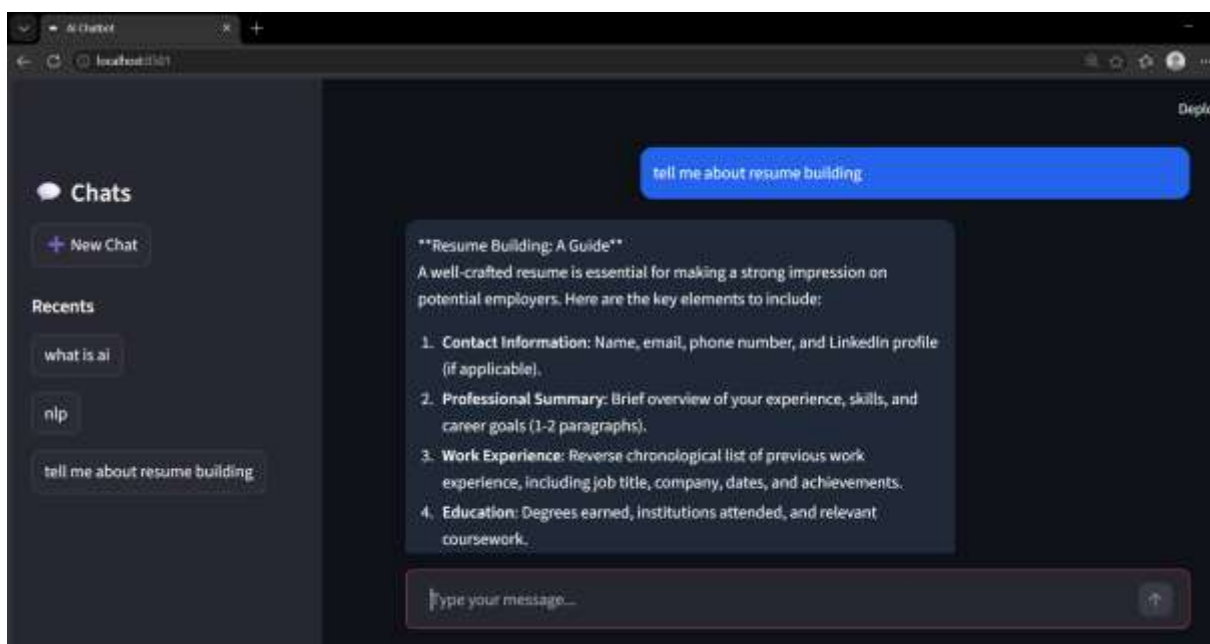
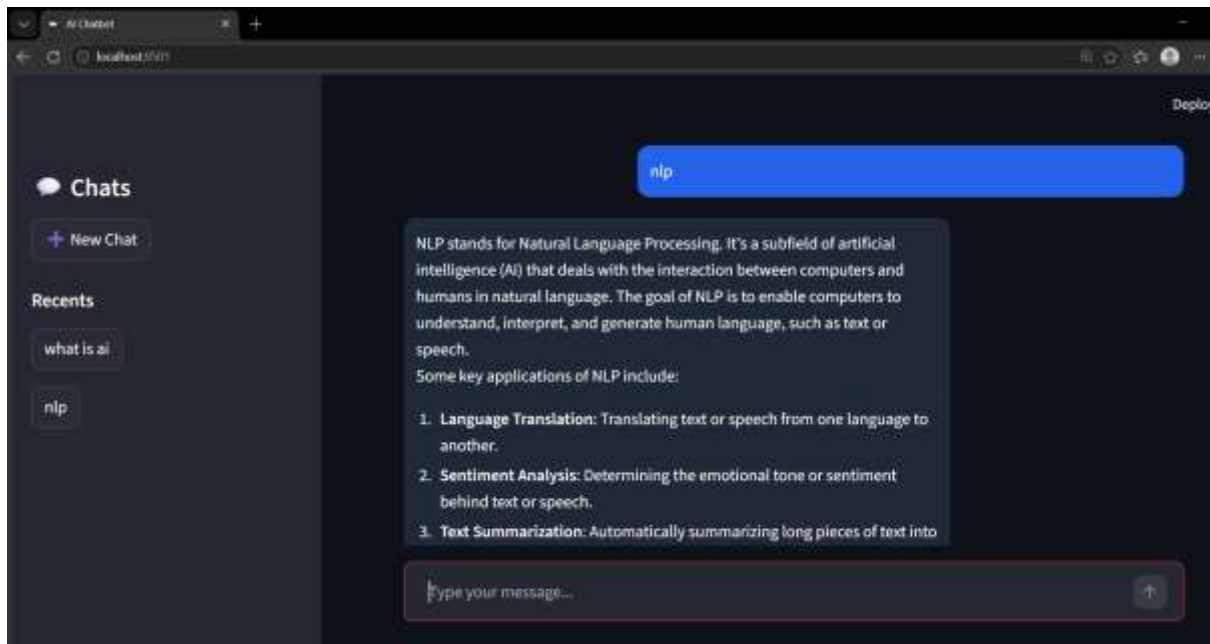
except Exception as e:

```
st.error(str(e))
```

```
st.rerun()
```

7.Output





8. Conclusion

This project demonstrates the implementation of a modern AI chatbot using LangChain and Groq's LLaMA model. The system highlights the use of prompt chaining, session management, and efficient UI design to create an interactive conversational application. The modular structure allows for easy scalability and future enhancements, making it a strong foundation for advanced AI-driven systems.